

* 전체 코드

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #define MAX 100
4
5  typedef struct in
6  {
7      int h;
8      int w;
9      int d;
10 }node;
11
12 int top;
13 int x, y;
14 int dd = 0;
15 int i = 2;
16
17
18 void init_stack(void)
19 {
20     top = -1;
21 }
22
23 node stack[MAX];
24
25 void push(int h, int w, int d)
26 {
27     if (top >= MAX - 1)
28     {
29         printf("Stack overflow\n");
30         return;
31     }
32     top++;
33     stack[top].h = x;
34     stack[top].w = y;
35     stack[top].d = dd;
36 }
37
38 node pop()
39 {
40     if (top < 0)
41     {
42         printf("Stack underflow\n");
43         node err = { -1,-1,-1 };
44         return err;
45     }
46     return stack[top--];
47 }
48
```

```

46     return stack[top--];
47 }
48
49 int input_map[15][15] = {
50     {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0},
51     {0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0},
52     {0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0},
53     {0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0},
54     {0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0},
55     {0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0},
56     {0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0},
57     {0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0},
58     {0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0},
59     {0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0},
60     {0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0},
61     {0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0},
62     {0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0},
63     {0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0},
64     {0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0}
65 };
66
67 int algorithm() {
68     int x1[8] = { 0, 1, 1, 1, 0, -1, -1, -1 };
69     int y1[8] = { -1, 1, 0, -1, 1, -1, 0, 1 };
70     int n;
71
72     for (n = 0; n < 8; n++) {
73         int x2 = x + x1[n];
74         int y2 = y + y1[n];
75
76         if (x2 >= 0 && x2 < 15 && y2 >= 0 && y2 < 15) {
77             if (input_map[x2][y2] == 1) {
78                 x = x2;
79                 y = y2;
80                 return n;
81             }
82         }
83     }
84     return -1;
85 }
86
87 int main() {
88     init_stack();
89     node temp;
90     int a;
91     int b;
92     for (int i = 0; i < 15; i++) {
93         for (int j = 0; j < 15; j++) {
94             printf("%d ", input_map[i][j]);
95         }
96         printf("\n");
97     }
98 }

```

```

88     init_stack();
89     node temp;
90     int a;
91     int b;
92     for (int i = 0; i < 15; i++) {
93         for (int j = 0; j < 15; j++) {
94             printf("%d ", input_map[i][j]);
95         }
96         printf("\n");
97     }
98
99     for (a = 0; a < 15; a++) {
100         for (b = 0; b < 15; b++) {
101             if (input_map[a][b] == 1) {
102                 x = a; y = b;
103                 input_map[x][y] = i;
104                 push(x, y, 0);
105
106                 while (top != -1) {
107                     dd = algorithm();
108
109                     if (dd != -1) {
110                         input_map[x][y] = i;
111                         push(x, y, dd);
112                     }
113                     else {
114                         temp = pop();
115                         x = temp.h;
116                         y = temp.w;
117                     }
118                 }
119             }
120         }
121     }
122     printf("\n\n");
123     printf("-----\n");
124     for (int i = 0; i < 15; i++) {
125         for (int j = 0; j < 15; j++) {
126             printf("%d ", input_map[i][j]);
127         }
128         printf("\n");
129     }
130 }

```

* 실행 결과

```
0 0 0 0 0 0 0 0 0 0 0 0 1 1 0
0 1 1 1 1 1 1 0 0 0 0 0 1 1 0
0 1 0 0 0 0 1 0 0 0 0 1 1 0 1 0
0 1 0 1 1 0 1 0 1 1 1 0 0 1 0
0 1 0 0 1 0 1 0 1 0 0 0 0 1 0
0 1 1 0 1 0 1 0 1 0 0 1 0 1 0
0 0 0 1 1 0 1 0 1 0 1 1 0 1 0
0 1 0 1 1 0 1 0 1 0 1 0 0 1 0
0 1 0 0 0 0 0 0 1 0 1 0 0 1 0
0 1 0 0 0 0 0 1 1 0 1 0 0 1 0
0 1 1 1 1 1 0 1 0 0 1 1 0 1 0
0 0 0 0 0 1 0 1 0 1 0 0 0 1 0
0 1 1 1 0 1 0 1 0 0 0 0 0 1 0
0 1 0 0 0 1 0 1 1 1 1 1 1 1 0
0 1 1 1 1 1 0 0 0 0 0 0 0 0 0

-----
0 0 0 0 0 0 0 0 0 0 0 0 2 2 0
0 2 2 2 2 2 2 0 0 0 0 0 2 2 0
0 2 0 0 0 0 2 0 0 0 0 2 2 0 2 0
0 2 0 2 2 0 2 0 2 2 2 0 0 2 0
0 2 0 0 2 0 2 0 2 0 0 0 0 2 0
0 2 2 0 2 0 2 0 2 0 0 2 0 2 0
0 0 0 2 2 0 2 0 2 0 2 2 0 2 0
0 2 0 2 2 0 2 0 2 0 2 0 0 2 0
0 2 0 0 0 0 0 0 2 0 2 0 0 2 0
0 2 0 0 0 0 0 2 2 0 2 0 0 2 0
0 2 2 2 2 2 0 2 0 0 2 2 0 2 0
0 0 0 0 0 2 0 2 0 2 0 0 0 2 0
0 2 2 2 0 2 0 2 0 0 0 0 0 2 0
0 2 0 0 0 2 0 2 2 2 2 2 2 2 0
0 2 2 2 2 2 0 0 0 0 0 0 0 0 0
```

* 코드 분석

1. 코드 동작 원리

이 코드는 왼쪽에서 오른쪽, 위에서 아래로 스캔한 후, 1을 발견하면 주변 8방향을 조사해서 또 다른 1이 있으면 그곳으로 이동하고, 현재 위치와 방향을 stack에 저장한다. 이동할때마다 번호를 매기고 주변에 더 이상 1이 없으면 스택에서 가장 최근에 저장했던 장소를 꺼내서 그 직전 위치로 되돌아간다. 결국 스택이 완전히 빌 때까지 주변 조사를 진행하는 것이다. 이렇게 15x15를 모두 조사하여 스택을 이용해서 번호를 매긴다.

2. 코드 구성 요소

Stack: h(행), w(열), d(방향)을 세트로 저장한다. 가장 마지막에 저장한 좌표를 가장 먼저 꺼내어 되돌아간다.

algorithm(): 현재 위치에서 주변 8방향 중 1이 있는지 찾는다. x1, y1 배열을 이용해 8방향을 검사하고, 길을 찾으면 그 방향 번호를 주고 x, y를 직접 수정한다.

3. 코드 시뮬레이션

- (1,1)에서 스캔을 시작, 8방향을 검사
- 방향 3에서 1을 발견, (1,1,1)을 스택에 저장
- 이것을 반복한 후 주변에서 1이 발견되지 않으면 pop()을 통해 전 위치로 복귀
- 스택이 비워지면 하나의 덩어리 탐색 종료

4. 후기

이 코드를 짤 때 main함수 외에도 정의해야할 함수들이 많아서 코드의 구조를 짜는 것에서 어려움을 겪었다. 특히 반복문 여러 개가 중첩되어 사용되었기 때문에 단순히 코드를 짜는 것으로 끝내지않고, 코드가 어떻게 실행되는지, 언제 어떤 코드가 작동해야하는지를 하나하나 체크하고 순서를 따져가며 코드를 작성했다. 결국 main 함수에는 최소한의 코드를 쓰는 방향으로 하고, 서로 다른 역할을 하는 다양한 함수를 main 함수 바깥쪽에 선언하는 방식을 선택하여 코드를 짜게 되었다. 이 과정에서 전역변수와 지역변수를 확실히 구분하는 법이 중요하다는 것을 알게되었고 여러 기능들을 서로 다른 함수로 모듈화하는 것이 중요하다는 것을 깨달았다.